Analyzing Network Traffic to Identify Plaintext Credentials Using tcpdump

Abstract

Network security is a critical aspect of modern computing, as sensitive data is constantly transmitted over networks. This project focuses on capturing and analyzing network traffic to identify whether sensitive information such as usernames and passwords can be exposed during transmission. Using the Linux command-line tool **tcpdump**, network packets were captured for both HTTP and HTTPS communication. The captured data was analyzed using command-line filters and the grep utility. The results demonstrate that HTTP traffic transmits data in plaintext, making credentials visible to attackers, whereas HTTPS traffic encrypts data, preventing exposure of sensitive information. This project highlights the importance of secure communication protocols in protecting user data.

1. Introduction

With the rapid growth of the internet and network-based applications, securing data during transmission has become a major concern. Many cyberattacks exploit unencrypted network traffic to steal sensitive information such as usernames, passwords, and session tokens. Packet sniffing is a common technique used by attackers to intercept network communication.

This project aims to practically demonstrate how insecure protocols like HTTP can expose sensitive information, while secure protocols like HTTPS protect data using encryption. By capturing and analyzing real network traffic, this project provides hands-on understanding of network vulnerabilities and secure communication practices.

2. Objectives

The objectives of this project are:

- To capture live network traffic using tcpdump
- To analyze HTTP traffic and identify plaintext credentials
- To analyze HTTPS traffic and verify that sensitive data is encrypted
- To understand the security implications of using unencrypted protocols

3. Tools and Environment

Component	Description
Operating System	Kali Linux
Packet Capture Tool	tcpdump
Analysis Tool	grep
Network Interface	eth0
HTTP Test Website	http://testphp.vulnweb.com/login.php
HTTPS Website	https://google.com

Lalanthika S

4. Methodology

4.1. Network Interface Identification

The active network interface was identified using the following command:

```
ip a
```

From the output, the eth0 interface was found to be active and assigned an IP address (192.168.1.14), indicating it was connected to the local network.

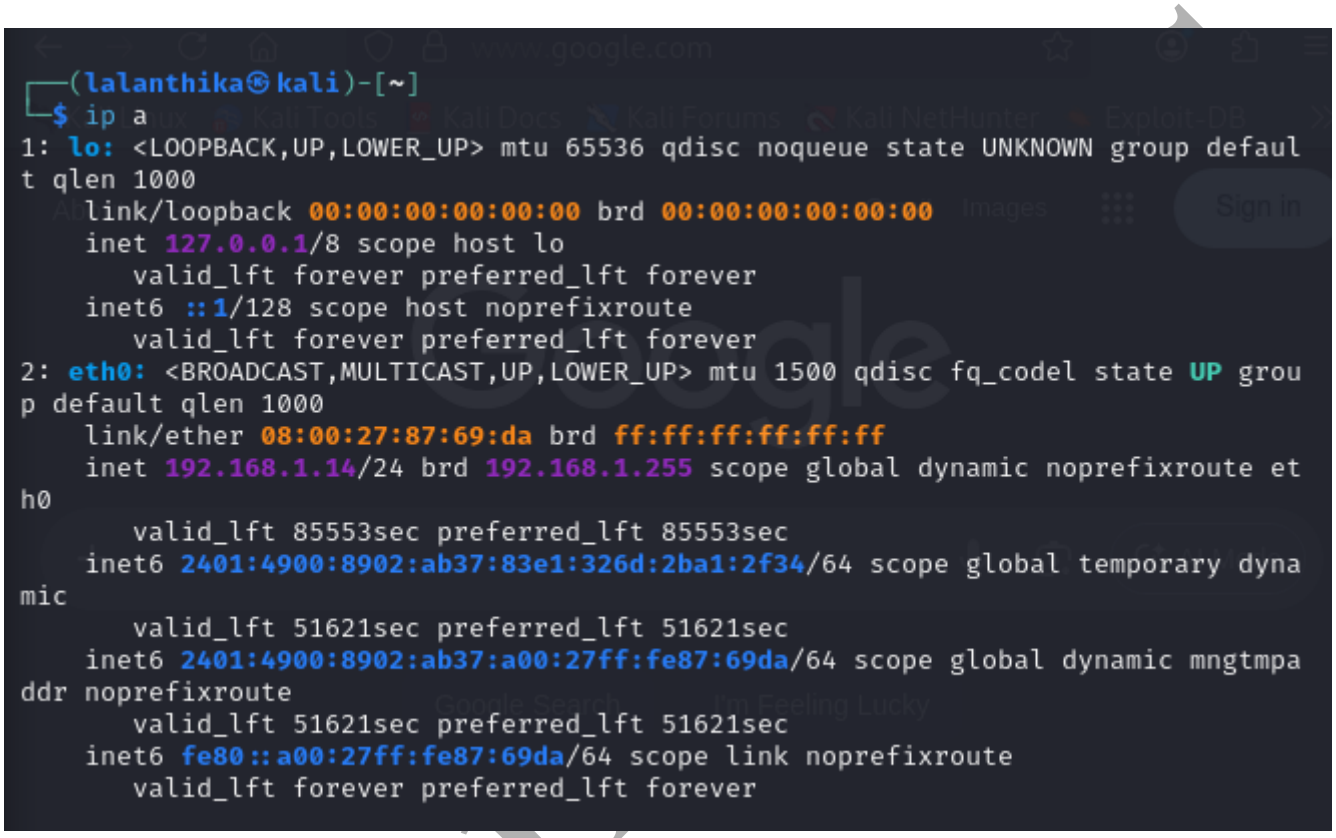


Fig: 4.1 – Listing all network interfaces on the system to identify the active interface for packet capture (eth0).

4.2. Capturing HTTP Traffic

To capture HTTP traffic, tcpdump was executed with root privileges:

```
sudo tcpdump -i eth0 -w project_http.pcap
```

While the capture was running, a login was performed on the HTTP test website using sample credentials. After completing the login, the capture was stopped using CTRL + C.



Fig: 4.2 - Capturing HTTP network traffic on the eth0 interface and saving it to project_http.pcap.

4.3. Analyzing HTTP Traffic

The captured HTTP packets were analyzed using:

```
tcpdump -r project_http.pcap -A 'tcp port 80'
```

```
┌──(lalanthika㉿kali)-[~]
└─$ tcpdump -r project_http.pcap -A 'tcp port 80'
reading from file project_http.pcap, link-type EN10MB (Ethernet), snapshot length
22:48:47.848249 IP6 2401:4900:8902:ab37:83e1:326d:2ba1:2f34.42664 > pnmaaa-aq-in-
val 2314096295 ecr 0,nop,wscale 8], length 0
`
...(.@$.I....7..2m+./4$.h.@..... ....P`..p..... ..........
..N.....
22:48:48.104469 IP 192.168.1.14.42976 > maa05s14-in-f3.1e100.net.http: Flags [S],
e 8], length 0
E..< ..@.@..1......LC...P>.7#.....".....
..Ta.....
22:48:48.154620 IP maa05s14-in-f3.1e100.net.http > 192.168.1.14.42976: Flags [S.]
 ecr 4106441825,nop,wscale 8], length 0
E..< ..@.{.b...LC.....P......>.7$.............
4.....Ta....
22:48:48.154656 IP 192.168.1.14.42976 > maa05s14-in-f3.1e100.net.http: Flags [.],
E..4..@.@..8......LC...P>.7$.............
..T.4...
22:48:48.157149 IP 192.168.1.14.42976 > maa05s14-in-f3.1e100.net.http: Flags [P.]
 434: HTTP: POST /s/wr3/y4Y HTTP/1.1
E.....@.@.........LC...P>.7$.............
..T.4...POST /s/wr3/y4Y HTTP/1.1
Host: o.pki.goog
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/ocsp-request
Content-Length: 84
Connection: keep-alive
Priority: u=2
Pragma: no-cache
Cache-Control: no-cache
```

Fig: 4.3 – Reading and displaying captured HTTP packets in ASCII format to observe transmitted data.

To search for sensitive information such as usernames and passwords, the output was filtered using grep:

```
tcpdump -r project_http.pcap -A 'tcp port 80' | grep -i pass
```

```
┌──(lalanthika㉿kali)-[~]
└─$ tcpdump -r project_http.pcap -A 'tcp port 80' | grep -i pass
reading from file project_http.pcap, link-type EN10MB (Ethernet), snapshot length
uname=test+&pass=
uname=test&pass=test
```

Fig: 4.4 – Filtering captured HTTP packets to locate plaintext passwords transmitted over the network.

The analysis revealed that credentials were transmitted in plaintext and were clearly visible in the captured packets.

4.4. Capturing HTTPS Traffic

Next, HTTPS traffic was captured using:

```
sudo tcpdump -i eth0 -w project_https.pcap
```

```
┌──(lalanthika㉿kali)-[~]
└─$ sudo tcpdump -i eth0 -w project_https.pcap

[sudo] password for lalanthika:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 b
ytes
^C8 packets captured
8 packets received by filter
0 packets dropped by kernel
```

Fig: 4.5 – Capturing HTTPS network traffic on the eth0 interface and saving it to project_https.pcap.

During the capture, a secure HTTPS website was accessed. The capture was stopped after a few seconds.

4.5. Analyzing HTTPS Traffic

The HTTPS capture file was analyzed using similar commands:

```
tcpdump -r project_https.pcap -A 'tcp port 443' | grep -i user
tcpdump -r project_https.pcap -A 'tcp port 443' | grep -i pass
```

```
┌──(lalanthika㉿kali)-[~]
└─$ tcpdump -r project_https.pcap -A 'tcp port 443' | grep -i pass
reading from file project_https.pcap, link-type EN10MB (Ethernet), snapshot lengt
h 262144

┌──(lalanthika㉿kali)-[~]
└─$ tcpdump -r project_https.pcap -A 'tcp port 443' | grep -i unme
reading from file project_https.pcap, link-type EN10MB (Ethernet), snapshot lengt
h 262144

┌──(lalanthika㉿kali)-[~]
└─$ tcpdump -r project_https.pcap -A 'tcp port 443' | grep -i uname
reading from file project_https.pcap, link-type EN10MB (Ethernet), snapshot lengt
h 262144
```

Fig: 4.6 – Searching captured HTTPS packets for usernames and password; encrypted payload prevents reading sensitive information.

No readable credentials were found, as the data was encrypted.

5. Network Flow Diagram

The diagram illustrates how network traffic flows from websites to a user's computer through a router. HTTP traffic is transmitted in plaintext, allowing sensitive information such as usernames and passwords to be captured by tcpdump. In contrast, HTTPS traffic is encrypted with SSL/TLS, making the captured packets unreadable and secure. This visualization clearly demonstrates the difference between unencrypted and encrypted communication and emphasizes the importance of using HTTPS for protecting sensitive data.

Lalanthika S

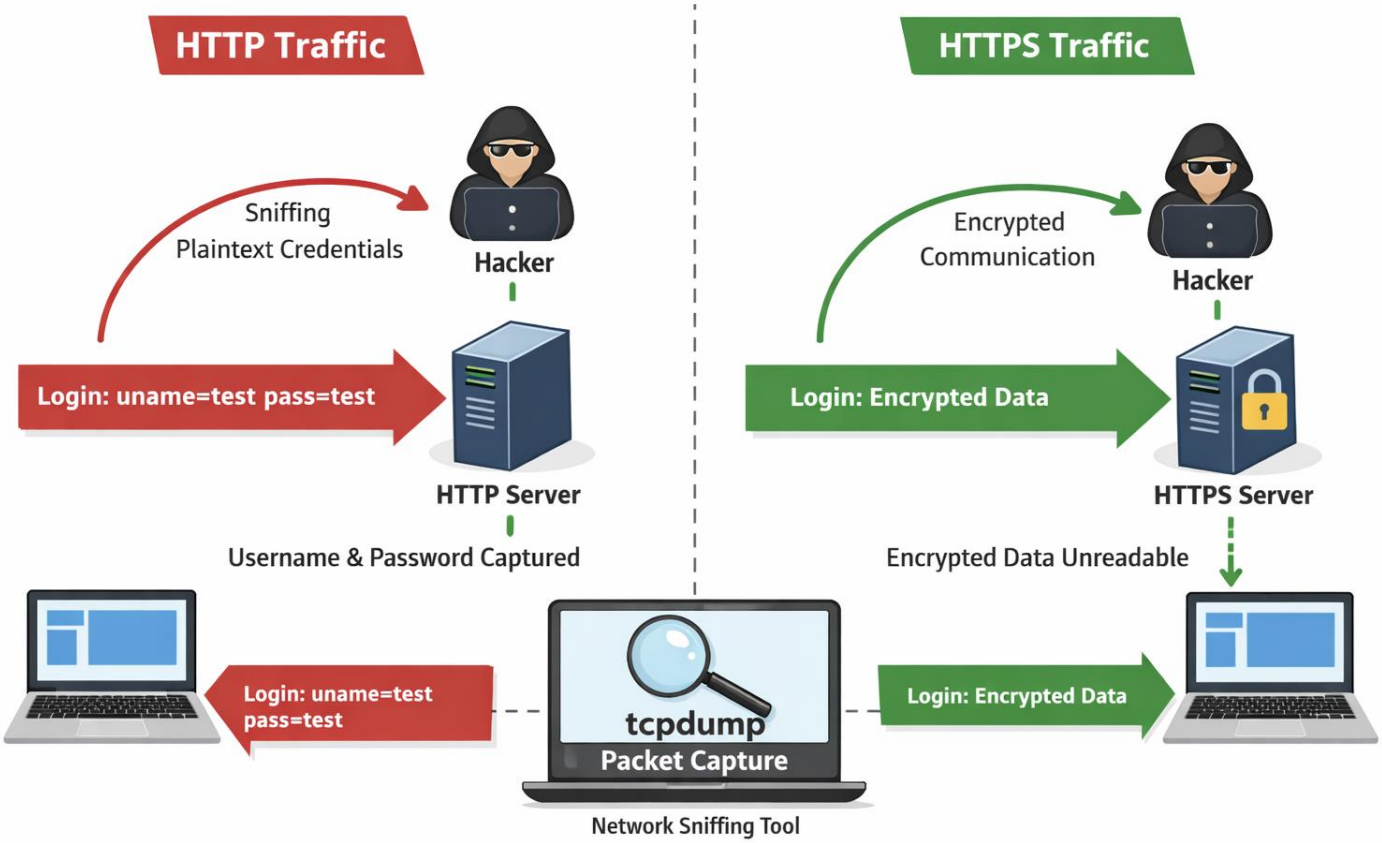


Fig: 5.1 – Network flow showing HTTP credentials exposed in plaintext versus HTTPS traffic protected by encryption.

6. Observations and Results

Protocol	Capture File	Result
HTTP	project_http.pcap	Username and password visible in plaintext
HTTPS	project_https.pcap	No readable sensitive information

Additional background traffic such as ARP, DNS, and mDNS packets were also observed, which is normal in a local network environment.

7. Analysis and Discussion

HTTP transmits data without encryption, making it vulnerable to packet sniffing attacks. Any attacker on the same network can capture packets and extract sensitive information using simple tools.

HTTPS, on the other hand, uses SSL/TLS encryption to secure data during transmission. Even though packets can still be captured, the encrypted payload prevents attackers from understanding the content.

This experiment clearly demonstrates why HTTPS is essential for protecting user credentials and why HTTP should not be used for sensitive communication.

Lalan

8. Security and Ethical Considerations

Packet capturing should only be performed in authorized environments such as personal labs or test networks. Capturing traffic without permission is illegal and unethical. This project was conducted strictly for educational purposes on permitted systems.

9. Conclusion

This project successfully demonstrated the difference between insecure and secure network communication. By capturing and analyzing HTTP and HTTPS traffic using tcpdump, it was observed that HTTP exposes sensitive information in plaintext, while HTTPS protects data through encryption. The project emphasizes the importance of secure protocols and provides practical insight into network traffic analysis and cybersecurity fundamentals.

10. Future Enhancements

- Perform graphical packet analysis using Wireshark
- Analyze other insecure protocols such as FTP or Telnet
- Study SSL/TLS handshakes in detail
- Implement intrusion detection techniques

11. References

1. tcpdump Official Documentation – https://www.tcpdump.org
2. OWASP WebGoat & VulnWeb – http://testphp.vulnweb.com
3. Computer Networking – Kurose & Ross

Lalanthika S